
A Comparison of Supervised Learning Architectures for Protein Sequence-to-Fitness Prediction Tasks

Gaurav Awasthi

MIT Department of Chemical Engineering
awasthi7@mit.edu

Zahmiria Bluesky Johnson

MIT Department of Chemical Engineering
zahmiria@mit.edu

1 Introduction

Protein design and engineering has revolutionized industry and biomedical research, with the potential to enhance sustainability^[1], therapeutics^[2], and target previously considered "undruggable" proteins^[3]. Experimental approaches such as directed evolution (DE) typically involve an iterative process of introducing diversity, screening for the top candidates, and then introducing further diversity to those^[1]. This process is time- and labor-intensive and can only sample a tiny fraction of the combinatorial sequence space. This complexity has led to machine learning (ML)-driven approaches, which have shown increasing promise on protein design and optimization problems^[4]. The task of accelerating protein engineering through *in silico* mutagenesis implicitly includes the task of predicting mutation effects, which is a longstanding challenge because protein fitness landscapes are rugged, high-dimensional, and shaped by complex, non-additive interactions between residues^[5]. The two broad paradigms through which ML models infer the sequence-structure-fitness relationship are model-based and de novo approaches^[6]. Model-based optimization typically involves learning the local fitness landscape around a template protein (often the wild type, WT) and then using an optimization algorithm to identify local fitness maxima. Models of varying complexity have been used to predict the effects of mutations on biomolecular function, including unsupervised^{[7], [8], [9]}, self-supervised^{[10], [11]}, and supervised^{[12], [13], [14]}. Conversely, de novo design uses information about the target sequence or structure to generate potential candidates fulfilling those criteria, potentially generalizing to novel mutational space that would not likely be explored experimentally. While advantageous in some contexts, de novo designs may suffer from expression or solubility issues and require further experimental optimization before therapeutic use is possible^[15].

This project focuses on model-based optimization using supervised learning architectures. Specifically, logistic regression and convolutional neural networks (CNNs) have been used to classify mutation effects^{[12], [16]}, as have transformers^{[17], [11]}. Separate from purely supervised models, the finetuning of protein and antibody language models on smaller experimental datasets has been shown to enhance performance on a variety of tasks^{[18], [19], [20], [21]}. This project aims to compare the performance of transformers with a CNN built by Thomas et al.^[16] to classify enzyme activity. We hypothesized that the attention mechanism would better enable the transformer to learn interactions between multiple residues. Prior work demonstrates that CNNs can successfully learn local sequence-function relationships^[22], and Yang et al.^[23] interestingly found CNNs to be competitive with transformers when masked language modeling was used for pretraining models on large protein datasets. We further benchmark those architectures against a logistic regression classifier that uses ESM-2^[24] embeddings of the authors' dataset.

We have attempted to evaluate the models along multiple axes. First, we compared the loss on the training and test datasets. Second, we analyzed performance as a function of number of mutations within the sequence to determine where these models diverge or converge as we proceed towards higher order mutations and epistatic interactions. We additionally probed the ESM-2 model to better understand its predictions. Ultimately, success was defined not only as improved predictive accuracy, but also a clearer understanding of when and why different model classes succeed in modeling nonlinear residue interactions in protein mutational landscapes. Architecture details, such as hyperparameter ranges, and some extended analyses are presented in the Appendix.

2 Methods

Our code is accessible at https://github.com/zahmiriajoh/AwasthiJohnson_10.C51.

2.1 Data sources

The dataset used in this study was generated by Thomas *et al.* across multiple rounds of a ML-guided DE campaign for the NucB endonuclease^[16]. The distributional plots shown in Figure 1 and the data used in training, validation, and testing are from a compilation of all generations, with cleaning to select only the sequences with known correct activity class labels. Following the authors' approach, variants with two or fewer mutations were used for training, while the remainder formed the test set. The dataset includes variants with up to 23 mutations (Figure 2). This split is designed to evaluate each model's ability to extrapolate to higher-order mutational combinations, a characteristic of DE campaigns.

The initial library (G1) was created by applying error-prone PCR (epPCR) to a single-site saturation mutagenesis library of the wild-type NucB sequence. Subsequent generations (G2, G3, G4) were designed using the TeleProt framework, which blends evolutionary data from natural NucB homologs with experimental fitness measurements accumulated across rounds. In addition to the ML-designed libraries, each generation also included hit recombination (HR) sub-libraries, which served as a non-ML baseline by recombining top-performing variants from the previous round. One key distinction is that throughout the simulation of DE, the test sets included erroneous sequences, as this is a given aspect of biological campaigns. The dataset described and used in this study excludes these errors to allow for focused evaluation on the performance of the models. Across all generations, the authors released a combined dataset of 55,760 variants—represented as 142 amino acid (AA) strings—with discrete activity labels (relative to the WT and A73R variant). A73R—indicating a WT sequence with the native Alanine at position 73 mutated to an Arginine—was identified as the best-performing variant in the experimentally-derived G1 library.

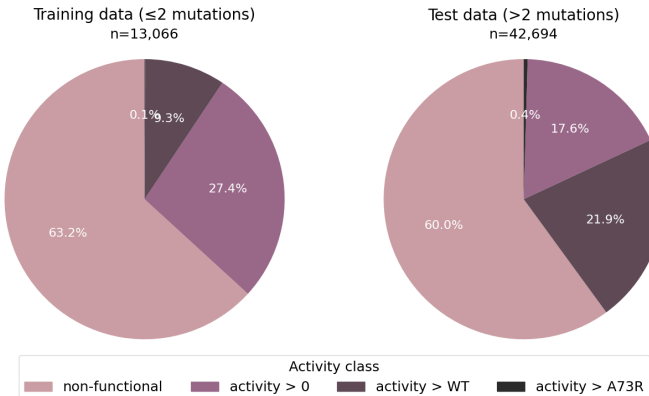


Figure 1: The full dataset split by mutation count for training and validation includes different percentages of activity classes.

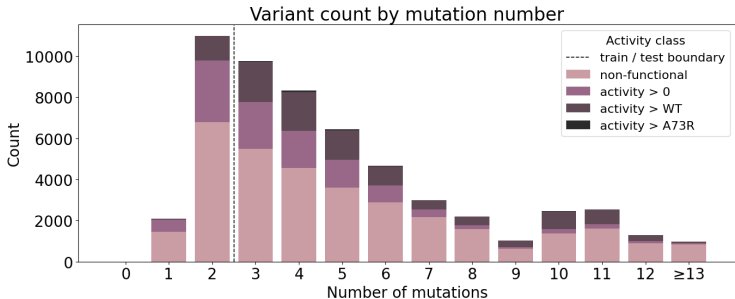


Figure 2: Training and testing splits are by mutation count in order to simulate predicting variants with more mutations for improving activity.

For the CNN, protein sequences were encoded as one-hot vectors. The transformer maps each amino acid to a learned 128-dimensional dense vector, then adds sinusoidal positional encodings so each residue’s representation carries both its identity and its position in the sequence.

2.2 Transformer classifier for comparison to CNN

2.2.1 Architecture

A transformer encoder was created to classify NucB variant sequences into the same four activity classes as the CNN. The architecture was designed as a direct analogue of the 3-layer CNN: where the CNN uses three width-5 convolutional layers to capture local sequence context, the transformer replaces these with three transformer encoder block layers that operate via global self-attention. Each encoder block follows a pre-norm design; layer normalization is applied before both the multi-head self-attention sublayer and the position-wise feed-forward network (FFN), with residual connections around each. The FFN uses GELU activations and has dimensionality $d_{\text{ffn}} = 2 \times d_{\text{model}}$. Input sequences, encoded as integer token indices over the 20 canonical amino acids, are first passed through a learned embedding layer of dimension d_{model} , followed by sinusoidal positional encodings to inject information about residue position. After the final encoder block, a layer normalization is applied and the per-residue representations are mean-pooled across the sequence dimension to yield a single d_{model} -dimensional vector, which is passed to a linear classification head to produce logits over the four activity classes. This mean-pooling step is analogous to the flatten-and-dense operation in the CNN. Raw logits are used during training with cross-entropy loss; softmax probabilities are computed externally at inference.

2.2.2 Handling class imbalance

Non-functional variants dominate the dataset, meaning that a model could achieve high nominal accuracy simply by predicting the majority class for every variant. We addressed this issue using weighted activity loss, which computes inverse-frequency class weights from the training label distribution as $w_c = N / (K \cdot n_c)$, where N is the total number of training examples, $K = 4$ is the number of classes, and n_c is the count of class c . These weights are passed to a cross entropy loss function so that the rare high-activity classes (*activity* > WT and *activity* > A73R) contribute proportionally more to the training signal.

2.2.3 Optimization

Both models—CNN and transformer—were trained using the AdamW optimizer with a learning rate schedule consisting of a linear warmup followed by cosine decay. Gradient norms were clipped to prevent instability. The models were trained for a fixed number of epochs, with the best checkpoint selected based on validation accuracy.

2.2.4 Hyperparameter selection

Hyperparameters were selected using an Optuna sweep with the Tree-structured Parzen Estimator (TPE) sampler, maximizing validation hit rate for *activity* > A73R as the primary objective. Median pruning was applied to terminate unpromising trials early, with a warmup of 5 epochs before pruning was eligible. The FFN dimension was held fixed at $2 \times d_{\text{model}}$, and the number of layers was included as a categorical hyperparameter with 3 as one of the candidate values, matching the CNN’s depth. The positional encoding was kept sinusoidal rather than learned for model simplicity and to avoid convoluting the comparison between model accuracy. An identical sweeping procedure was applied for both the transformer and the CNN in order to maintain equivalent evaluation, with a constant batch size of 256 and 50 epochs in training. Importantly, the train/val split was kept identical between the transformer and CNN sweeps—using the same `mutation_count_split` function with the same seed and parameters—ensuring that the models are compared on the same held-out validation set without data leakage. The swept hyperparameters are summarized in Appendix A Table 2 and Table 3.

2.3 Classification using learned representations from ESM-2

2.3.1 Sequence representations

We extracted protein sequence representations using ESM-2^[24], a transformer-based protein language model pretrained on UniRef50^[25] via masked language modeling. To minimize computational requirements, we used the two smallest publicly available variants, `esm2_t6_8M_UR50D` (link) and `esm2_t12_35M_UR50D` (link). The former model contains 6 transformer layers, 320-dimensional hidden states, and approximately 8 million parameters, while the latter has 12 transformer layers, 480-dimensional hidden states, and approximately 35 million parameters; both models were accessed via the HuggingFace `transformers` library. These are hereafter referred to as the ‘8M ESM model’ and ‘35M ESM model’ respectively.

Train and test datasets were constructed in the same way as above. For the WT and each variant, we passed the full 142-residue sequence through the encoder in inference mode (no gradient computation) and obtained per-residue contextual embeddings from the final hidden layer. `AutoModel` (link) was used to load only the encoder part of the ESM-2 model since we don’t need the masked-language modeling part. A batch size of 8 sequences was fed to the model in a single forward pass. Some elements of this workflow were adapted from a publicly available tutorial (link) on the ESM-2 HuggingFace page.

ESM-2 wraps each sequence with a start and end token, making all sequences 144 tokens long. Before mean-pooling, we manually masked out these two tokens so that the resulting fixed-length representation reflects only the 142 residue positions. The last hidden state of the model has shape $(N_{\text{batch}}, N_{\text{tokens}}, N_{\text{model dim}})$, where $N_{\text{batch}} = 8$, $N_{\text{tokens}} = 144$, and $N_{\text{model dim}} = 320$ or 480 depending on which ESM-2 variant is used. We then performed mean-pooling across the 142 residue positions (dropping the padded start and end tokens) to compute a 320- or 480-dimensional representation per sequence.

2.3.2 Logistic regression top model

A multinomial logistic regression top model was trained to learn the mapping from these sequence representations to the four enzyme activity classes labelled in the authors’ dataset. This approach was largely inspired by that of Biswas et al.^[18]; we used the frozen embeddings from ESM-2 instead of finetuning the model to reduce the computational requirements. The mean-pooled embeddings were standardized (mean 0, variance 1) and then passed to a multinomial logistic regression classifier with L2 regularization, implemented through the scikit-learn library. The `StandardScaler` was fit only on the training data and with the same parameters was applied to the test data. 5-fold cross-validation was performed to select the L2 regularization strength (inverse strengths of 0.001, 0.01, 0.1, 1.0, 10.0, 100.0, 1000.0, and 10000.0 were tested). $\lambda = 0.001$ was chosen as the optimal regularization strength for the 8M ESM model. Each individual fit minimizes the regularized categorical cross-entropy loss (cross-entropy + L2 penalty). The choice among candidate regularization strengths was made by macro F1 score on held-out folds, which weights all four activity classes equally regardless of their prevalence.

2.4 Scores and hits@1000 calculation

Cumulative scores were also calculated in the same way as Thomas et al.^[16]:

$$\text{score}_{\text{WT}}(x) = P(\text{activity} > \text{WT} \mid x) + P(\text{activity} > \text{A73R} \mid x)$$

$$\text{score}_{\text{A73R}}(x) = P(\text{activity} > \text{A73R} \mid x)$$

The `hits@1000` metric was calculated by ranking the test sequences of the relevant class by their score, and reporting the fraction of the top 1000 that were genuine hits. The authors used this metric as it reflects the design use case that the classifier is intended for: the intention of the *in silico* directed evolution campaign is to have confident predictions of the top-ranked candidates. Thus, the primary metrics for comparison of the models in this study was through training loss and `hits@1000` on the test data; the latter used at activity thresholds corresponding to the WT and A73R. As the test set composition shows (Figure 1), the `activity > WT` class is 22% of the test set, while the `activity > A73R` class is 0.4% of the test set.

3 Results

3.1 Model performance comparison

3.1.1 Loss, hit rate, and accuracy

The CNN and the transformer had comparable loss over the training loop. The loss curves in Figure 3A show that both the CNN and transformer stabilize but with some degree of overfitting as the transformer loss decreases with approximately constant validation loss leading up to the final epochs. The ESM-2 iteration curve was not included on this plot as we felt that the iterations of the LR optimizer were not directly comparable to the epochs of a CNN or transformer. The final training loss of the ESM-2 classifier was 0.59, lower than both.

The three models—CNN, transformer, and ESM—are primarily compared and optimized by the hits@1000 metric for variants better than the WT and better than the A73R mutant (Figure 3B). None of the models achieved notable hits@1000_{A73R}, with the CNN performing best at only 0.014 — roughly three times the random baseline of 0.004 but far from practically useful. For the less stringent activity > WT threshold, the ESM model achieved the highest hits@1000_{WT} of 0.311. This could suggest that the pretrained representations encode sequence-function information relevant to moderate activity improvements but that even rich evolutionary embeddings fall short when tasked with identifying the rare, highly-active variants that would be most valuable in a real DE campaign (at least with a simple logistic regression classifier).

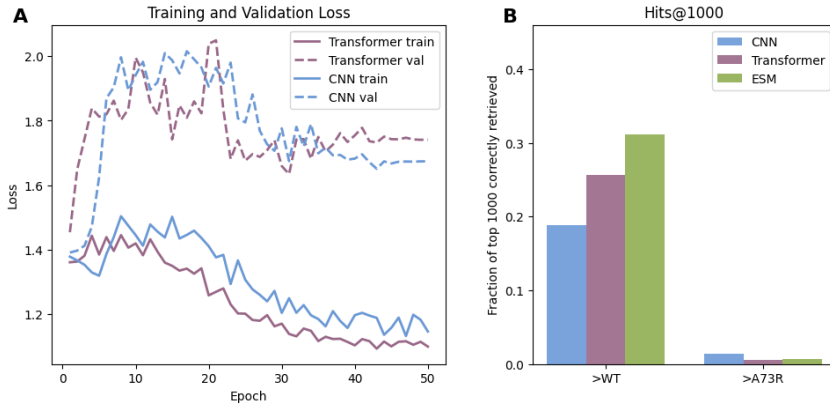


Figure 3: The models are optimized and compared by evaluation of the loss (A) and the hits@1000 metric (B).

The confusion matrices in Figure 4 reveal a shared failure mode across all three models: high-activity variants are frequently misclassified into adjacent lower-activity classes, with the activity > A73R class being the most difficult to recover.

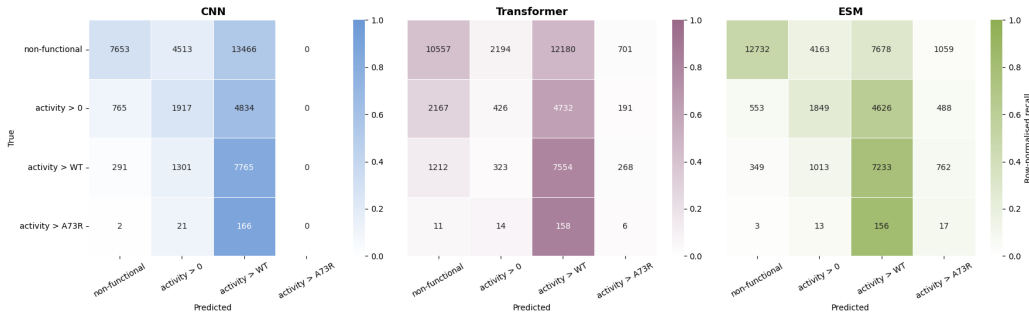


Figure 4: The three models display similar patterns in classification, as shown by confusion matrices.

3.1.2 Performance across mutation number

To summarize performance across mutation counts, we formulated a High Activity Misclassification Rate (HAMR) metric, defined as the proportion of variants with true activity > WT that are misclassified as non-functional (Figure 5A). This metric tracks a particularly high-risk failure mode in the context of DE: the loss of potent variants to the non-functional bin, which would typically be discarded without further investigation or experimental validation. We plot HAMR across increasing mutation count as an approximation of a DE campaign, where successive rounds of mutagenesis progressively add mutations (Figure 5B). The CNN and transformer both show decreasing HAMR with increasing mutation count and converge towards zero at higher mutation numbers. The CNN maintains the lowest HAMR across all bins, representing the most desirable trajectory for a model intended to guide library design. The ESM model shows the opposite trend, with HAMR increasing at higher mutation counts, implying that the pretrained representations become less reliable as variants diverge further from the wild type.

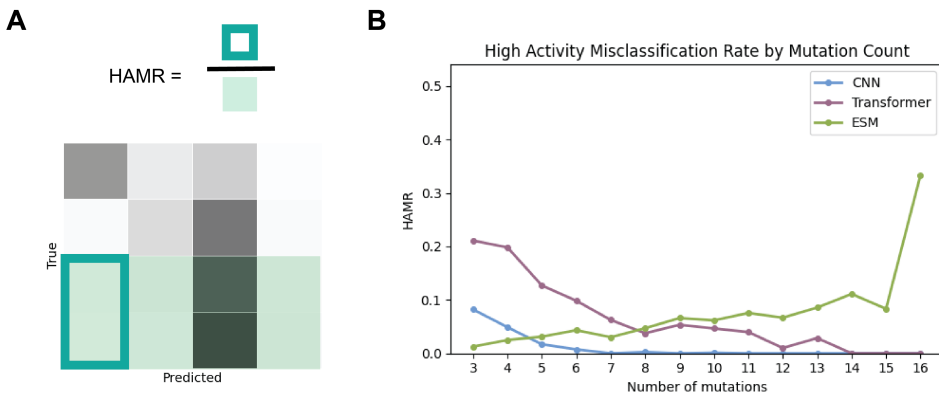


Figure 5: A. The HAMR metric is the proportion of high activity variants that are misclassified as non-functional. B. HAMR conveys different trends with mutation count for each model.

3.2 Probing ESM representations

Despite using a simple classifier like multinomial logistic regression, the 8M ESM-2 model achieved better training loss and showed superior performance (on the hits@1000 metric) to the one-hot encoded transformer and CNN. In the following subsections, we probe the ESM-2 representations along various axes to better understand their performance and attempt to improve it.

Effect of ESM-2 model size

Following the same procedure as described above, we tested the 35M ESM-2 model and noticed marginal improvement (Table 1). It is likely that testing even larger ESM-2 variants would lead to a further improvement; however, to avoid using further computational resources and to focus on other analyses, we decided to forego that study.

Comparing the effect of balancing class weights on the hits@1000 metric

Balancing class weights is standard practice in imbalanced classification problems as it weights each training example inversely to the frequency of its class, ensuring the model learns to perform well on each class. With only 10 activity > A73R examples in the train data, the rare class is upweighted significantly and the classifier is incentivized to assign high probabilities to any sequence that resembles this class, but learns a poor decision boundary anchored on only 10 points. These falsely confident predictions may improve the overall cross-entropy loss, but will decrease the hits@1000 metric. Both models were re-trained on an unweighted cross-entropy loss (Table 1). Marginal changes were observed in for the 8M ESM-2 model, but there was marked improvement for the 35M ESM-2 model. To ensure the models stayed comparable with the CNN and the transformer, and to stay true to the original intention, the balanced classes were preserved in downstream analysis.

Model	Class weighting	hits@1000 _{WT}	hits@1000 _{A73R}
ESM-2 8M	balanced	0.311	0.007
	unweighted	0.315	0.007
ESM-2 35M	balanced	0.305	0.009
	unweighted	0.346	0.012

Table 1: Ranking-quality metrics across ESM-2 model size and class-weighting configurations.

Projection of ESM embeddings

Since increasing model size and changing the class weights did not lead to a marked improvement in model predictions, we attempted to visualize the actual ESM-2 representations of the sequences to check for their difference. ESM-2 is trained on a large section of all known proteins; our dataset, consisting of mutations to one enzyme, represents a very small fraction of that. Without sufficiently different embeddings, logistic regression may be unable to separate the classes. Figure 6 shows a whole-landscape t-SNE projection of all 55760 variants colored by their activity class, with the WT and A73R specifically highlighted to visualize the distance between them. A randomly chosen non-functional variant A33C has also been highlighted for comparison. The 8M ESM model shows no clear geometric separation between the classes. Wondering if this was an artifact of model size, we plotted the results for the 35M ESM model and still observed no significant separation.

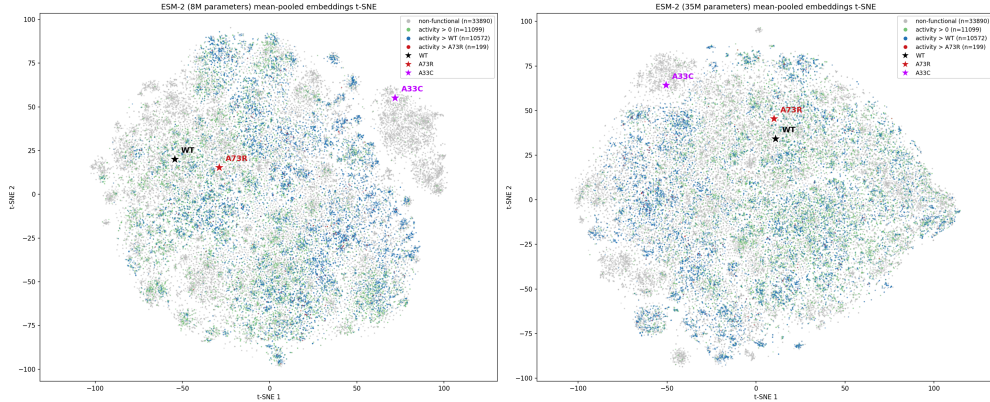


Figure 6: t-SNE projections of the mean-pooled representations of all sequences in the dataset corresponding to the 8M (left) and 35M (right) ESM-2 models. WT, A73R (functional, better than WT) and A33C (non-functional) highlighted.

We hypothesized that the mean pooling performed to collapse the per-residue embeddings into a sequence-level embedding may be causing the diffuse structure of the embedding space: since more than half the sequences have ≤ 5 mutations in a 142-residue protein, it is possible that the signal from the mutated positions is attenuated upon mean pooling. This hypothesis was further probed in Appendix B.

Replacing mean pooling with max pooling after taking the difference from the WT representation

To evaluate the hypothesis of "dilution" of signal, we retrained the model by first subtracting the 142-dimensional representation of the WT from each sequence, and then performing a max-pooling operation on the absolute values. By preserving the difference from the WT, we were hopeful that the models would better learn the features responsible for changing activity, and max pooling would ideally focus on the most important features. Figure 7 shows relatively better separation between the classes than Figure 6. Even with balanced weights, the 8M model attained a hits@1000_{WT}=0.348 and hits@1000_{A73R}=0.012, better than the mean-pooled performance of even the 35M model (Table 1).

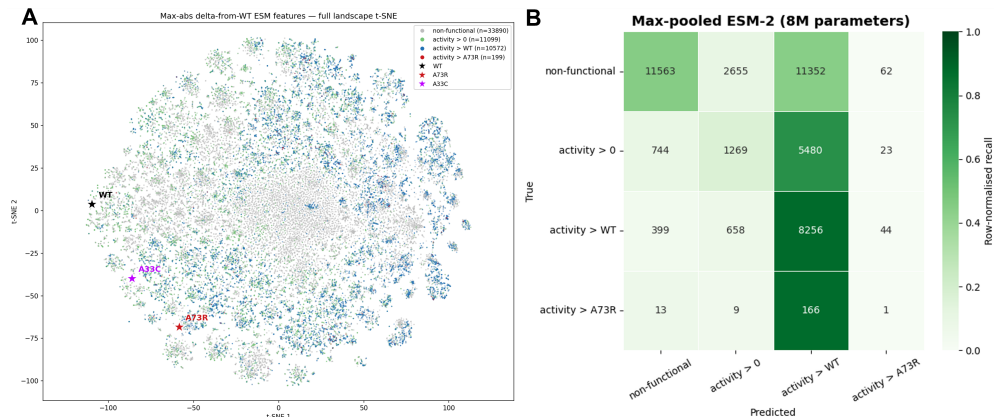


Figure 7: (A) t-SNE projection of the max-pooled representations of 8M ESM-2 model after subtracting the WT representations. (B) Confusion matrix for the max-pooled ESM-2 model evaluated on the test set.

4 Conclusion and future work

This study compared three supervised learning architectures — a CNN, a transformer, and a logistic regression classifier on ESM-2 embeddings — for the task of predicting NucB enzyme activity from sequence, using a mutation-count-based train/test split to simulate extrapolation across rounds of directed evolution. All three models outperformed random baselines on the hits@1000 metric, yet none achieved practically useful performance at the hardest threshold of activity > A73R, highlighting the fundamental difficulty of identifying rare, highly-active variants from a small and imbalanced training set. The CNN and transformer performed comparably to each other, with the ESM-2 model performing better despite using a simple logistic regression classifier. Performance was further improved by replacing mean-pooling with a WT-difference max-pooling strategy, suggesting that how pretrained embeddings are aggregated matters significantly. The HAMR analysis further revealed a qualitative divergence between the models: the CNN and transformer become more reliable at higher mutation counts, whereas ESM-2 degraded in the high-mutation regime most relevant to later rounds of directed evolution. Together, these results suggest that for small, enzyme-specific datasets, transformers can be implemented to classify variant activity to comparable performance to CNNs but without strong evidence towards improved performance. Further, general-purpose protein language model embeddings used out of the box remain competitive with locally-trained models on simple encodings.

Several directions remain for future work. The most immediate extension would be to finetune ESM-2 end-to-end on the NucB dataset rather than using frozen embeddings; while computationally more expensive, updating the representations to be task-specific would likely improve hits@1000 performance and may produce better-separated embedding geometries. A more rigorous epistatic analysis would also be valuable, beginning with an additive baseline that assumes independent mutational effects and progressively adding complexity through a supervised Potts model capable of capturing first-order non-additivity^[7]. Though, the random nature of epPCR in G1 may complicate inference for mutations that never appeared in isolation, and a more structured dataset generated by site-saturation mutagenesis may be better suited to such a study^[26]. Finally, the framework could be extended beyond enzymes to proteins with less evolutionary conservation, such as antibodies, where the hyper-variable CDR regions that govern binding are poorly represented in general protein databases; in that setting, comparing general language models like ESM against antibody-specific models such as AbLang^[27] and antiBERTy^[28] would test whether domain-specific pretraining corpora outperform broad evolutionary information for fitness prediction.

5 Contributions

ZJ implemented the transformer and CNN with the associated contributions to analysis and writing. ZJ collaborated with Claude Sonnet 4.6 to code the transformer, to replicate the CNN, and to create the repository structure. Claude was used for summarizing and trimming written materials but not for ideation. GA performed literature review, project conceptualization and worked specifically on all tasks related to the use of ESM-2 and resulting analyses. GA used Claude Opus 4.7 for assistance with programming tasks, but did not use any AI tools for literature review, ideation or writing related to this project.

References

- [1] Philip A. Romero and Frances H. Arnold. “Exploring Protein Fitness Landscapes by Directed Evolution”. In: *Nature Reviews Molecular Cell Biology* 10.12 (2009), pp. 866–876 (cit. on p. 1).
- [2] Nathaniel R. Bennett et al. “Atomically Accurate de Novo Design of Antibodies with RFdiffusion”. In: *Nature* 649.8095 (2026), pp. 183–193 (cit. on p. 1).
- [3] Leo Tianlai Chen et al. “Target Sequence-Conditioned Design of Peptide Binders Using Masked Language Modeling”. In: *Nature Biotechnology* (2025), pp. 1–9 (cit. on p. 1).
- [4] Bruce J. Wittmann, Yisong Yue, and Frances H. Arnold. “Informed Training Set Design Enables Efficient Machine Learning-Assisted Directed Protein Evolution”. In: *Cell Systems* 12.11 (2021), 1026–1045.e7 (cit. on p. 1).
- [5] Kevin K. Yang, Zachary Wu, and Frances H. Arnold. “Machine-Learning-Guided Directed Evolution for Protein Engineering”. In: *Nature Methods* 16.8 (2019), pp. 687–694 (cit. on p. 1).
- [6] Jiayao Zheng et al. “The Application of Machine Learning on Antibody Discovery and Optimization”. In: *Molecules* 29.24 (2024), p. 5923 (cit. on p. 1).
- [7] Thomas A. Hopf et al. “Mutation Effects Predicted from Sequence Co-Variation”. In: *Nature Biotechnology* 35.2 (2017), pp. 128–135 (cit. on pp. 1, 8).
- [8] Thomas A Hopf et al. “The EVCouplings Python Framework for Coevolutionary Sequence Analysis”. In: *Bioinformatics* 35.9 (2019), pp. 1582–1584 (cit. on p. 1).
- [9] Adam J. Riesselman, John B. Ingraham, and Debora S. Marks. “Deep Generative Models of Genetic Variation Capture the Effects of Mutations”. In: *Nature Methods* 15.10 (2018), pp. 816–822 (cit. on p. 1).
- [10] Ethan C. Alley et al. “Unified Rational Protein Engineering with Sequence-Based Deep Representation Learning”. In: *Nature Methods* 16.12 (2019), pp. 1315–1322 (cit. on p. 1).
- [11] Alexander Rives et al. “Biological Structure and Function Emerge from Scaling Unsupervised Learning to 250 Million Protein Sequences”. In: *Proceedings of the National Academy of Sciences* 118.15 (2021), e2016239118 (cit. on p. 1).
- [12] Steffanie Paul et al. *Machine Learning Enables Efficient and Effective Affinity Maturation of Nanobodies*. 2026 (cit. on p. 1).
- [13] Derek M. Mason et al. “Optimization of Therapeutic Antibodies by Predicting Antigen Specificity from Antibody Sequence via Deep Learning”. In: *Nature Biomedical Engineering* 5.6 (2021), pp. 600–612 (cit. on p. 1).
- [14] Emily K. Makowski et al. “Co-Optimization of Therapeutic Antibody Affinity and Specificity Using Machine Learning Models That Generalize to Novel Mutational Space”. In: *Nature Communications* 13.1 (2022), p. 3788 (cit. on p. 1).
- [15] Jie Gao et al. “De Novo Protein Design: A Transformative Frontier in Clinical Protein Applications”. In: *Journal of Translational Medicine* 24.1 (2026), p. 319 (cit. on p. 1).
- [16] Neil Thomas et al. “Engineering Highly Active Nuclease Enzymes with Machine Learning and High-Throughput Screening”. In: *Cell Systems* 16.3 (2025), p. 101236 (cit. on pp. 1, 2, 4, 12).
- [17] Roshan Rao et al. “Evaluating Protein Transfer Learning with TAPE”. In: *Advances in neural information processing systems* 32 (2019), pp. 9689–9701 (cit. on p. 1).
- [18] Surojit Biswas et al. “Low-N Protein Engineering with Data-Efficient Deep Learning”. In: *Nature Methods* 18.4 (2021), pp. 389–396 (cit. on pp. 1, 4).

- [19] Chloe Hsu et al. “Learning Protein Fitness Models from Evolutionary and Assay-Labeled Data”. In: *Nature Biotechnology* 40.7 (2022), pp. 1114–1122 (cit. on p. 1).
- [20] Robert Schmirler, Michael Heinzinger, and Burkhard Rost. “Fine-Tuning Protein Language Models Boosts Predictions across Diverse Tasks”. In: *Nature Communications* 15 (2024), p. 7407 (cit. on p. 1).
- [21] Meng Wang et al. “Supervised Fine-Tuning of Pre-Trained Antibody Language Models Improves Antigen Specificity Prediction”. In: *PLOS Computational Biology* 21.3 (2025), e1012153 (cit. on p. 1).
- [22] Michael N Pun et al. “Learning the shape of protein microenvironments with a holographic convolutional neural network”. In: *Proceedings of the National Academy of Sciences* 121.6 (2024), e2300838121 (cit. on p. 1).
- [23] Kevin K. Yang, Nicolo Fusi, and Alex X. Lu. “Convolutions Are Competitive with Transformers for Protein Sequence Pretraining”. In: *Cell Systems* 15.3 (2024), 286–294.e2 (cit. on p. 1).
- [24] Zeming Lin et al. “Evolutionary-Scale Prediction of Atomic-Level Protein Structure with a Language Model”. In: *Science* 379.6637 (2023), pp. 1123–1130 (cit. on pp. 1, 4).
- [25] Baris E. Suzek et al. “UniRef: Comprehensive and Non-Redundant UniProt Reference Clusters”. In: *Bioinformatics* 23.10 (2007), pp. 1282–1288 (cit. on p. 4).
- [26] Emily E. Wrenbeck et al. “Plasmid-Based One-Pot Saturation Mutagenesis”. In: *Nature Methods* 13.11 (2016), pp. 928–930 (cit. on p. 8).
- [27] Tobias H Olsen, Iain H Moal, and Charlotte M Deane. “AbLang: An Antibody Language Model for Completing Antibody Sequences”. In: *Bioinformatics Advances* 2.1 (2022), vbac046 (cit. on p. 8).
- [28] Jeffrey A. Ruffolo, Jeffrey J. Gray, and Jeremias Sulam. *Deciphering Antibody Affinity Maturation with Language Models and Weakly Supervised Learning*. 2021. arXiv: 2112.07782 [q-bio] (cit. on p. 8).

A Transformer and CNN architecture

	Parameter	Range	Selected value
Optimization	learning rate	$[10^{-4}, 10^{-3}]$ (log scale)	0.001
	weight decay	$[10^{-5}, 10^{-1}]$ (log scale)	2.14E-2
Architecture	d_model	64, 128, 256	128
	num_heads	2, 4, 8	2
	num_layers	2, 3, 4, 6	3
	dropout	[0.0, 0.2]	0.004

Table 2: Structure and results of the hyperparameter sweep for the transformer.

	Parameter	Range	Selected value
Optimization	learning rate	$[10^{-4}, 10^{-3}]$ (log scale)	1.52E-4
	weight decay	$[10^{-5}, 10^{-1}]$ (log scale)	5.42E-5
Architecture	num_filters	16, 32, 64	32
	num_conv_layers	2, 3, 4, 6	3
	kernel_size	3, 5, 9	3
	hidden_dim	64, 128, 256	128
	dropout	[0.0, 0.2]	0.042

Table 3: Structure and results of the hyperparameter sweep for the CNN.

B Per-residue embeddings of the ESM-2 models

To probe the effects of mean-pooling, we visualized the cosine distance between the per-residue embeddings of the WT and the two single-mutant sequences (A73R and A33C). The results are displayed in Figure 8 for the 8M and 35M ESM models. We notice that the embeddings differ only at the mutated positions (particularly for A73R), which means that the signal may likely get attenuated upon mean-pooling. The mutation at residue 33, on the other hand, leads to a change in embedding in approximately a 5-amino acid radius around the position. Further, the cosine distance for almost all other positions is greater for A33C than for A73R. This makes it more likely that mean-pooling would still preserve some of the signal from this difference in embeddings, and this may be the reason why A33C is much more distant from the WT in the t-SNE plots (Figure 6) relative to A73R. Further, the peaks are wider for the 35M model than the 8M model, indicating that the richer representations better capture the effects of a point mutation on the surrounding residues. Biophysically, it may indicate that position 33 has more epistatic interactions with its neighbors than position 73; this may be verified through analysis of the enzyme structure. Interestingly, neither position seems to exhibit significant long-range interactions.

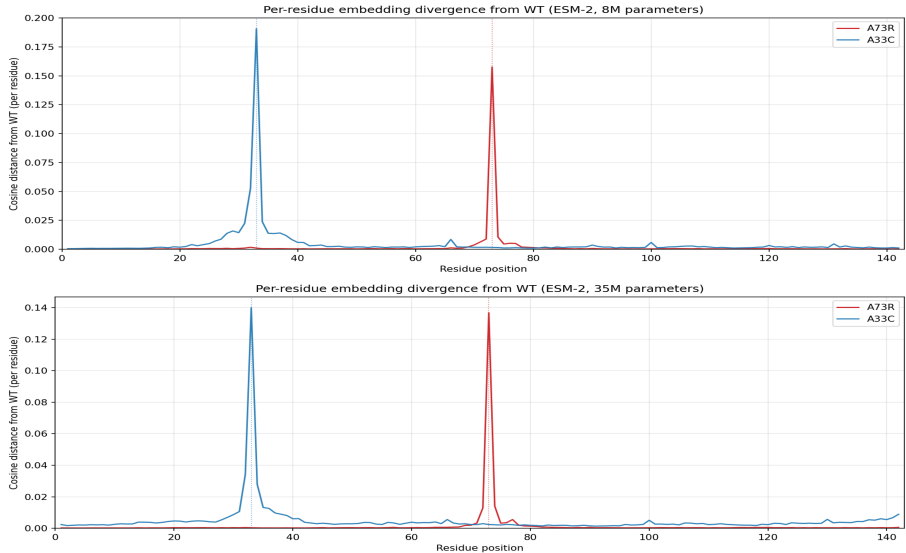


Figure 8: Per-residue cosine distance between the embeddings of WT and mutated sequences in the 8M (top) and 35M (bottom) ESM-2 models.

C Performance across mutation density splits

As a preliminary probe of the ESM model’s ability to generalize epistatic effects, we stratified performance according to the minimum distance between any two mutated residues in a triple mutant sequence. Distances are binned individually at the low end as we expected significant epistasis between neighboring residues due to amino acid charge and size effects. Figure 9 indicates a weak trend towards greater misclassification at shorter distances, which may be indicative of nonlinear epistatic effects confounding the model; a deeper analysis across mutation numbers would need to be performed before making conclusions. Confusion matrices corresponding to different mutation numbers and distances are accessible at the Github repository.

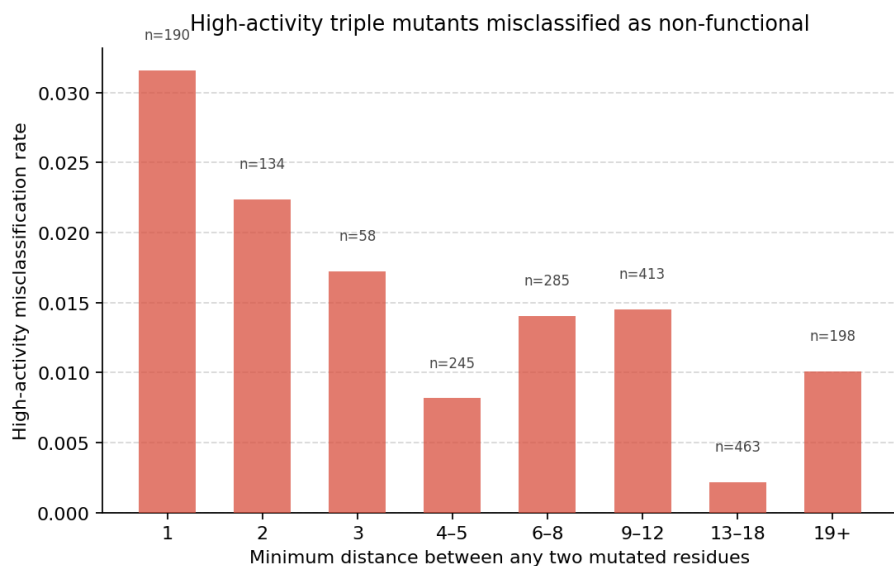


Figure 9: Proportion of high activity triple mutants misclassified as non-functional binned according to the minimum distance between two mutated residues. The 8M ESM-2 model was used.

D Generation-based split of training and test data (8M ESM-2 model)

In addition to the split based on number of mutations presented in the main text above, the 8M ESM-2 model was trained on a combined set of sequences from G1, G2 and G3, and then tested on G4; the model was thus exposed to a wide range of mutation numbers during training. This evaluates the model’s ability to predict the fitness of out-of-distribution, improved variants that would be seen in a directed evolution campaign. The authors tested a similar use of their model in Table S5^[16]. A confusion matrix was generated for different bins of mutation count; only the overall confusion matrix is shown here, and the rest can be found in the Github repository. Figure 10 shows mostly off-diagonal element mistakes, which means that the model predicts the wrong class by one but does not significantly misclassify high activity variants as non-functional, or vice-versa. The model’s recall on the active classes is seen to get weaker as the number of mutations increase, indicating that the model’s predictive ability for out-of-distribution variants is still limited (Table 4).

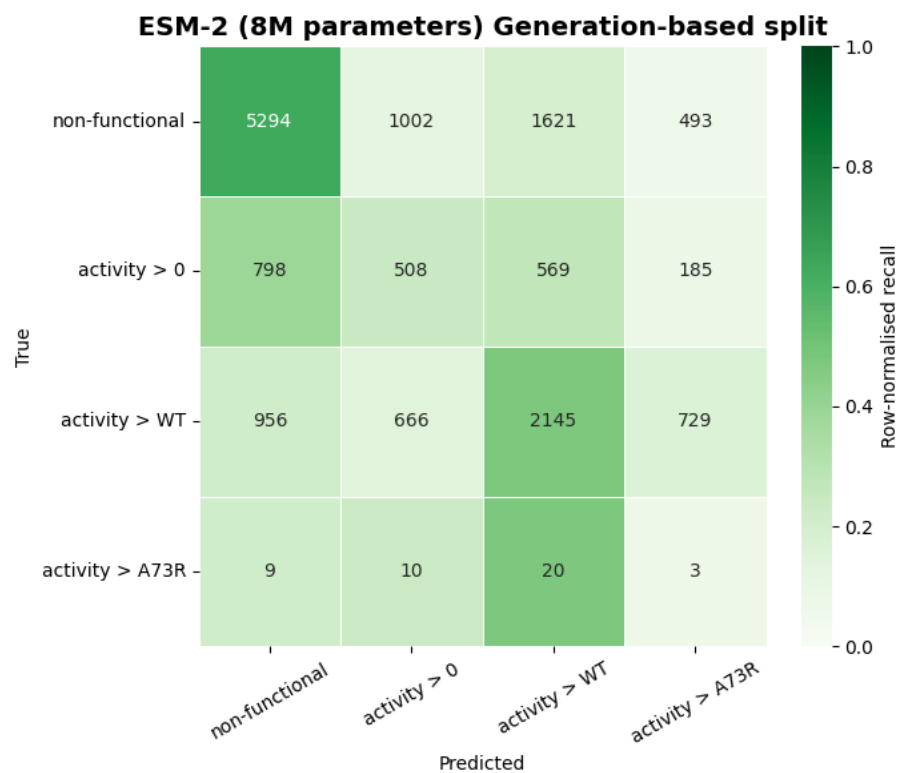


Figure 10: Confusion matrix for 8M ESM-2 model trained on G1+G2+G3 and tested on G4.

Mutation count	Number of samples	Non-functional recall	Activity>WT recall
2	754	0.59	0.33
3–5	2,805	0.68	0.45
6–10	6,622	0.54	0.49
11–15	4,521	0.68	0.49
16+	305	0.89	0.00

Table 4: 8M ESM-2 per-class recall stratified by mutation count from wild type.